

UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO DO UÍGE

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

TRABALHO GRUPO DE ALGORITMOS AVANÇADOS

TEMA: OTIMIZAÇÃO POR ENXAME DE PARTÍCULAS (PARTICLE SWARM OPTIMIZATION — PSO)

POR:

João Bosco K. Miguel

&

Simão Teca Panzo

Orientado Pelo Docente

Joaquim João Nsaku Ventura

UÍGE, NOVEMBRO DE 2026

RESUMO

A Otimização por Enxame de Partículas, conhecida como Particle Swarm Optimization (PSO), é uma meta-heurística estocástica de otimização populacional inspirada em aspetos do comportamento coletivo de grupos de animais, como bandos de aves e cardumes de peixes. Proposta por James Kennedy e Russell Eberhart em 1995, utiliza um conjunto de partículas que representam soluções candidatas e que atualizam as suas posições com base na experiência individual e na informação coletiva.

Este trabalho apresenta os fundamentos teóricos do PSO, os seus componentes, as equações de atualização, o pseudocódigo, uma implementação introdutória em Python, a análise da complexidade, aplicações, vantagens, limitações e uma proposta de avaliação experimental. O PSO pode ser útil em problemas em que não é necessário calcular derivadas da função objetivo, mas não garante, em geral, encontrar o ótimo global. A qualidade dos resultados depende da formulação do problema, dos parâmetros, da inicialização e das estratégias de pesquisa.

Palavras-chave: otimização; algoritmos avançados; inteligência de enxame; meta-heurística; Particle Swarm Optimization.

INTRODUÇÃO

A otimização é uma área importante da Matemática Aplicada, da Engenharia e da Ciência da Computação. Muitos problemas procuram minimizar custos, reduzir erros, maximizar eficiência ou selecionar a melhor configuração entre várias alternativas. Quando o espaço de pesquisa é grande, não linear ou complexo, a procura exaustiva pode ser demasiado dispendiosa.

As meta-heurísticas são estratégias gerais que orientam a pesquisa por soluções de boa qualidade sem exigir, necessariamente, uma enumeração de todas as alternativas. Uma dessas técnicas é a Otimização por Enxame de Partículas (PSO), apresentada por Kennedy e Eberhart em 1995. O algoritmo trabalha com uma população de partículas, cada uma representando uma solução candidata, e utiliza a experiência individual e coletiva para orientar a pesquisa.

O PSO é relativamente simples de implementar e pode ser aplicado a diferentes problemas, mas o seu desempenho não é garantido para todos os contextos. A convergência prematura, a sensibilidade aos parâmetros e a representação das soluções são aspetos que devem ser analisados com rigor.

1. INVESTIGAÇÃO E OBJETIVOS

1.1 Problema de investigação

Como o algoritmo Particle Swarm Optimization utiliza a interação entre partículas para procurar soluções de problemas de otimização, e que fatores influenciam a eficiência, a convergência e a qualidade dos resultados?

1.2 Objetivo geral

Estudar os fundamentos teóricos e matemáticos do PSO, compreender o seu funcionamento e analisar a sua implementação, aplicações e limitações.

1.3 Objetivos específicos

- Definir otimização, meta-heurísticas e inteligência de enxame.
- Apresentar a origem e a evolução do PSO.
- Explicar os componentes e parâmetros do algoritmo.
- Interpretar as equações de atualização da velocidade e da posição.
- Apresentar pseudocódigo e uma implementação introdutória em Python.
- Analisar a complexidade computacional e os critérios de paragem.
- Identificar aplicações, vantagens, limitações e variantes.
- Propor experiências reprodutíveis para avaliar o algoritmo.

1.4 Justificação

O tema permite relacionar matemática, algoritmia, programação e análise experimental. O PSO é especialmente útil para estudar como uma população de soluções pode ser orientada por informação individual e coletiva. A investigação também permite compreender que uma meta-heurística pode encontrar boas soluções sem garantir o ótimo global.

1.5 Metodologia

A metodologia proposta combina pesquisa bibliográfica, análise matemática e experimentação computacional. Devem ser consultados artigos científicos e documentação técnica. A implementação será testada numa função com ótimo conhecido e os resultados reais serão registados e analisados. Não se devem apresentar resultados hipotéticos como se fossem resultados de execução.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Conceito de otimização

Um problema geral de minimização pode ser representado por:

min f(x), sujeito a $x \in S$

Nesta expressão, x é uma solução candidata, S é o conjunto de soluções admissíveis e f(x) é a função objetivo. Num problema de maximização, procura-se o maior valor da função. As restrições definem quais as soluções aceitáveis.

2.2 Inteligência de Enxame

A Inteligência de Enxame estuda sistemas em que comportamentos coletivos organizados surgem da interação entre indivíduos relativamente simples. Em algoritmos computacionais, esta ideia é adaptada para coordenar a pesquisa de soluções. O PSO inspira-se em aspetos do comportamento coletivo, mas não reproduz integralmente um sistema biológico.

2.3 Origem e evolução

Kennedy e Eberhart apresentaram o PSO em 1995. Trabalhos posteriores desenvolveram variantes, incluindo formulações com peso de inércia, fator de constrição, vizinhanças locais, versões binárias e abordagens híbridas. As revisões de Cheng et al. (2018), Gad (2022) e Shami et al. (2022) descrevem a evolução e as áreas de aplicação do método.

2.4 Classificação

- Populacional: mantém várias soluções candidatas em simultâneo.
- Estocástico: utiliza números aleatórios e pode produzir resultados diferentes em execuções distintas.
- Meta-heurístico: fornece uma estratégia geral de pesquisa adaptável a vários problemas.
- Sem derivadas na formulação convencional: a atualização não exige o cálculo de derivadas da função objetivo.
- Iterativo: atualiza soluções ao longo de várias iterações.

3. COMPONENTES E FUNCIONAMENTO DO PSO

3.1 Posição

A posição da partícula i num problema com D variáveis é representada pelo vetor $x_i = [x_{i1}, x_{i2}, \ldots, x_iD]$. A posição representa a solução candidata atual.

3.2 Velocidade

A velocidade $v_i = [v_{i1}, v_{i2}, \ldots, v_iD]$ determina a alteração aplicada à posição entre iterações.

3.3 Melhor posição individual (pbest)

pbest é a melhor posição que a partícula encontrou até ao momento. Num problema de minimização, é atualizada quando a nova posição apresenta um valor menor da função objetivo.

3.4 Melhor posição global ou de vizinhança (gbest/lbest)

Na variante global, gbest é a melhor posição encontrada por qualquer partícula. Noutras topologias, cada partícula pode seguir a melhor posição da sua vizinhança, muitas vezes designada lbest.

3.5 Função objetivo

A função objetivo mede a qualidade de uma solução. Por exemplo, $f(x)=x^2$ tem mínimo global zero em x=0. A função e o sentido da otimização devem ser definidos antes de executar o PSO.

3.6 Etapas gerais

1. Definir a função objetivo, as variáveis, os limites e os critérios de paragem.
2. Inicializar as posições e velocidades das partículas.
3. Avaliar a função objetivo para cada partícula.
4. Inicializar e atualizar as melhores posições individuais.
5. Determinar a melhor posição global ou de vizinhança.
6. Atualizar as velocidades e posições.
7. Aplicar a estratégia de tratamento de limites e restrições.
8. Repetir até cumprir o critério de paragem.
9. Apresentar a melhor solução encontrada e o respetivo valor objetivo.

4. MODELO MATEMÁTICO

4.1 Atualização da velocidade

Na formulação com peso de inércia, a velocidade atualiza-se por:

$v_id(t+1) = w \cdot v_id(t) + c_1 \cdot r_1 \cdot [p_id(t) - x_id(t)] + c_2 \cdot r_2 \cdot [g_d(t) - x_id(t)]$

Em que w é o peso de inércia; c_1 e c_2 são coeficientes cognitivo e social; r_1 e r_2 são números aleatórios, normalmente em [0,1]; p_id é a melhor posição individual; g_d é a melhor posição de referência; e x_id é a posição atual.

4.2 Atualização da posição

A posição é atualizada através de:

$x_id(t+1) = x_id(t) + v_id(t+1)$

A equação adiciona a nova velocidade à posição anterior. Em problemas com limites, a posição resultante deve ser verificada e corrigida segundo uma estratégia documentada.

4.3 Interpretação dos parâmetros

O peso de inércia w controla a influência da velocidade anterior. c_1 controla a influência da melhor posição individual e c_2 controla a influência da melhor posição de referência. Valores mais elevados ou mais baixos alteram a dinâmica da pesquisa, mas não existe uma configuração universalmente ótima.

4.4 Exploração e aproveitamento

Exploração significa pesquisar regiões diferentes do espaço de soluções. Aproveitamento significa concentrar a pesquisa em regiões promissoras já identificadas. Um equilíbrio inadequado pode conduzir a convergência prematura ou a uma pesquisa pouco eficaz.

4.5 Restrições

As estratégias de tratamento dos limites incluem limitar posições, refletir o movimento, ajustar velocidades ou reparar soluções. A escolha afeta o comportamento do algoritmo e deve ser indicada no relatório.

5. PSEUDOCÓDIGO

```
Algoritmo PSO para minimização

Entrada: f(x), N partículas, D dimensões, T iterações,
         w, c1, c2 e limites das variáveis

Inicializar posições x[i] e velocidades v[i]
Para cada partícula i:
    pbest[i] = x[i]
    avaliar f(x[i])
Determinar gbest

Para t = 1 até T:
    Para cada partícula i:
        Para cada dimensão d:
            gerar r1 e r2 em [0,1]
            v[i,d] = w*v[i,d]
                     + c1*r1*(pbest[i,d]-x[i,d])
                     + c2*r2*(gbest[d]-x[i,d])
            x[i,d] = x[i,d] + v[i,d]
        corrigir limites, se necessário
        avaliar f(x[i])
        se f(x[i]) < f(pbest[i]):
            pbest[i] = x[i]
    atualizar gbest
    verificar critério de paragem

Devolver gbest e f(gbest)
```

O pseudocódigo representa uma versão global convencional. Uma implementação completa deve especificar a inicialização, o controlo da velocidade, a correção dos limites e a política de paragem.

6. IMPLEMENTAÇÃO INTRODUTÓRIA EM PYTHON

Para demonstrar o método, considera-se a função $f(x)=x^2$ no intervalo [-10,10]. O ótimo global conhecido é $x^*=0$ e $f(x^*)=0$. A implementação abaixo é educativa e utiliza uma variável.

```
import random

def funcao_objetivo(x):
    return x ** 2

def pso():
    numero_particulas = 30
    iteracoes = 100
    w = 0.7
    c1 = 1.5
    c2 = 1.5
    limite_inferior = -10
    limite_superior = 10

    posicoes = [
        random.uniform(limite_inferior, limite_superior)
        for _ in range(numero_particulas)
```

```
    ]
    velocidades = [
        random.uniform(-1, 1)
        for _ in range(numero_particulas)
    ]

    pbest = posicoes.copy()
    indice = min(
        range(numero_particulas),
        key=lambda i: funcao_objetivo(pbest[i])
    )
    gbest = pbest[indice]

    for _ in range(iteracoes):
        for i in range(numero_particulas):
            r1 = random.random()
            r2 = random.random()

            velocidades[i] = (
                w * velocidades[i]
                + c1 * r1 * (pbest[i] - posicoes[i])
                + c2 * r2 * (gbest - posicoes[i])
            )

            posicoes[i] += velocidades[i]

            if posicoes[i] < limite_inferior:
                posicoes[i] = limite_inferior
                velocidades[i] = 0
            elif posicoes[i] > limite_superior:
                posicoes[i] = limite_superior
                velocidades[i] = 0

            if funcao_objetivo(posicoes[i]) < funcao_objetivo(pbest[i]):
                pbest[i] = posicoes[i]

        indice = min(
            range(numero_particulas),
            key=lambda i: funcao_objetivo(pbest[i])
        )
        gbest = pbest[indice]

    return gbest, funcao_objetivo(gbest)

if __name__ == "__main__":
    melhor_x, melhor_valor = pso()
    print("Melhor solução encontrada:", melhor_x)
    print("Valor da função objetivo:", melhor_valor)
```

A saída depende dos números aleatórios. O resultado real deve ser obtido executando o programa; não se deve apresentar um valor hipotético como resultado experimental.

6.1 Explicação do programa

10. A função funcao_objetivo calcula x^2.
11. As posições e velocidades são inicializadas aleatoriamente.
12. pbest armazena a melhor posição individual de cada partícula.

13. gbest armazena a melhor posição conhecida pelo enxame.
14. Em cada iteração, o programa atualiza velocidade e posição.
15. As posições fora do intervalo são limitadas e a velocidade é anulada.
16. As melhores posições são atualizadas sempre que se encontra uma solução melhor.
17. No final, o programa apresenta a melhor posição e o seu valor objetivo.

7. COMPLEXIDADE COMPUTACIONAL

Se N for o número de partículas, D o número de dimensões e T o número de iterações, a atualização das partículas exige, em geral, O(NDT) operações. Esta estimativa assume custo constante para avaliar a função objetivo.

Se a avaliação da função tiver custo C_f, uma estimativa mais informativa é O(NT(D + C_f)), entendendo C_f como o custo computacional de uma avaliação. Em problemas que exigem simulações dispendiosas, o custo da função objetivo pode dominar o tempo total.

O armazenamento de posições, velocidades e melhores posições individuais requer, em geral, O(ND) memória auxiliar, sem contar dados adicionais de histórico, vizinhança ou resultados.

A complexidade não garante a qualidade da solução: custo computacional e capacidade de encontrar uma boa solução são aspetos diferentes.

8. APLICAÇÕES

Engenharia

O PSO pode procurar parâmetros que reduzam custos, erros ou melhorem o desempenho de sistemas.

Aprendizagem automática

Pode ser utilizado para procurar hiperparâmetros ou selecionar características, conforme a formulação do problema.

Redes de computadores

Pode apoiar problemas de encaminhamento, configuração e distribuição de recursos, se as soluções forem representadas adequadamente.

Escalonamento

Pode ser adaptado à atribuição de tarefas e recursos; problemas discretos exigem representação e operadores apropriados.

Energia e indústria

Pode procurar configurações de sistemas e parâmetros operacionais com base numa função objetivo definida.

Estas são áreas de aplicação estudadas na literatura, não uma garantia de que o PSO seja a técnica mais adequada para qualquer problema de cada domínio.

9. VANTAGENS E LIMITAÇÕES

9.1 Vantagens

- Estrutura matemática relativamente simples.
- Implementação acessível e adaptável.
- Pesquisa com várias soluções candidatas em simultâneo.
- A formulação convencional não exige derivadas da função objetivo.
- Existem variantes para diferentes classes de problemas.

9.2 Limitações

- Pode convergir prematuramente para uma região subótima.
- Não garante, em geral, encontrar o ótimo global.
- O desempenho depende da parametrização e da inicialização.
- Pode enfrentar dificuldades em espaços de elevada dimensão.
- Problemas discretos e combinatórios podem exigir variantes específicas.
- Resultados estocásticos devem ser avaliados através de várias execuções.

10. VARIANTES DO PSO

10.1 PSO com peso de inércia

Inclui w na equação da velocidade para controlar a influência do movimento anterior.

10.2 PSO com fator de constrição

Utiliza um fator adicional em determinadas formulações para controlar a dinâmica da velocidade e estudar propriedades de convergência.

10.3 PSO com vizinhança local

Cada partícula utiliza a melhor solução da sua vizinhança, em vez de seguir necessariamente a melhor solução de todo o enxame.

10.4 PSO binário

Destina-se a variáveis binárias e exige regras de atualização adequadas ao domínio discreto.

10.5 PSO híbrido

Combina PSO com outras estratégias para procurar melhorar a diversidade ou o aproveitamento. A superioridade de uma variante deve ser demonstrada para o problema considerado.

11. PROPOSTA DE AVALIAÇÃO EXPERIMENTAL

11.1 Experiência inicial

Minimizar $f(x)=x^2$ no intervalo [-10,10]. O ótimo conhecido é $x^*=0$, com valor zero. Devem registar-se os parâmetros, a versão do Python, o ambiente de execução e as sementes aleatórias.

11.2 Experiência multidimensional

Minimizar $f(x,y)=x^2+y^2$ com x e y em [-10,10]. O ótimo conhecido é (0,0), com valor zero.

11.3 Comparação de parâmetros

Comparar, por exemplo, w=0,4, w=0,7 e w=0,9, mantendo os restantes parâmetros e o protocolo experimental constantes. A conclusão deve basear-se nos resultados obtidos, não numa suposição antecipada.

11.4 Repetição e indicadores

Executar cada configuração com várias sementes aleatórias e registar o melhor valor, a média, o desvio-padrão, o tempo e o número de avaliações. Definir previamente uma tolerância para calcular a taxa de sucesso.

Os resultados experimentais só devem ser preenchidos depois de executar o código. Uma tabela vazia pode ser usada no plano experimental; valores simulados ou inventados não devem ser apresentados como observações reais.

CONCLUSÃO

A Otimização por Enxame de Partículas é uma meta-heurística populacional inspirada em aspetos do comportamento coletivo. A atualização das posições e velocidades permite às partículas combinar experiência individual e informação coletiva para procurar soluções de boa qualidade. O estudo dos componentes, das equações, do pseudocódigo e da implementação mostra a relação entre teoria e prática.

O PSO apresenta vantagens como simplicidade, adaptabilidade e ausência de necessidade de derivadas na formulação convencional. Contudo, não garante o ótimo global em todos os problemas e pode ser sensível aos parâmetros ou convergir prematuramente. A utilização responsável exige formulação adequada, implementação correta, experiências reprodutíveis e análise crítica dos resultados.

REFERÊNCIAS BIBLIOGRÁFICAS

Cheng, S., Lu, H., Lei, X., & Shi, Y. (2018). A quarter century of particle swarm optimization. Complex & Intelligent Systems, 4, 227–239. https://doi.org/10.1007/s40747-018-0071-2

Gad, A. G. (2022). Particle swarm optimization algorithm and its applications: A systematic review. Archives of Computational Methods in Engineering, 29, 2531–2561. https://doi.org/10.1007/s11831-021-09694-4

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. Proceedings of ICNN'95 — International Conference on Neural Networks, 1942–1948. IEEE. https://doi.org/10.1109/ICNN.1995.488968

Shami, T. M., El-Saleh, A. A., Alswaitti, M., Al-Tashi, Q., Summakieh, M., & Mirjalili, S. (2022). Particle swarm optimization: A comprehensive survey. IEEE Access, 10, 10031–10061. https://doi.org/10.1109/ACCESS.2022.3142859

Python Software Foundation. (s.d.). Python documentation. https://docs.python.org/3/

Microsoft. (s.d.). Python in Visual Studio Code. https://code.visualstudio.com/docs/languages/python

MathWorks. (s.d.). Particle swarm optimization algorithm. MATLAB Global Optimization Toolbox documentation. https://www.mathworks.com/help/gads/particle-swarm-optimization-algorithm.html